AgentTelemetry: An Open-Source Toolkit for Fault Detection in LLM Agent Systems

Krishna Chaitanya Balusu
Independent Researcher
United States
krishnabkc15@gmail.com

Abstract

Autonomous AI agents built on large language models (LLMs) fail in ways that standard distributed tracing cannot diagnose: reasoning loops, circular delegation, guardrail bypass, and planning failures all manifest as opaque `INTERNAL` spans in OpenTelemetry (OTel). We present AgentTelemetry, an open-source, pip-installable Python toolkit (3,700+ LOC, Apache-2.0) that extends OTel with nine agent-specific span kinds—including five novel kinds absent from the OTel GenAI semantic conventions—enabling automated fault detection for 14 agent failure types. The toolkit provides adapters for seven agent frameworks, four analysis modules, and a real-time circuit breaker. In a controlled evaluation across 2,940 configurations, AgentTelemetry achieves a fault detection rate (FDR) of 1.000 versus 0.429 for vanilla OTel. On 112 SWE-bench Lite instances, reasoning loops account for 75% of failures, and a telemetry-guided intervention improves the patch rate by +11 percentage points. Instrumentation overhead is 11.7 µs median per span.

Tool: https://github.com/agenttelemetry/agenttelemetry | `pip install agenttelemetry`
Video: https://youtu.be/PLACEHOLDER

CCS Concepts

• **Software and its engineering → Software verification and validation**; *Software post-development issues.*

Keywords

AI agents, observability, OpenTelemetry, fault detection, LLM monitoring, distributed tracing

ACM Reference Format:
Krishna Chaitanya Balusu. 2026. AgentTelemetry: An Open-Source Toolkit for Fault Detection in LLM Agent Systems. In . ACM, New York, NY, USA, 4 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

LLM-based autonomous agents—systems that pursue goals through iterative reasoning, planning, tool use, and delegation—are rapidly moving from research prototypes to production [1]. Frameworks such as LangChain, CrewAI, AutoGen, and LlamaIndex have accelerated adoption, but a critical gap remains: *there is no standardized observability infrastructure for diagnosing agent failures.*

OpenTelemetry (OTel) is the standard for distributed tracing in microservice architectures. Its GenAI semantic conventions define operation types for LLM calls, tool execution, retrieval, and agent lifecycle. However, five agent orchestration phases—planning, reasoning, safety-check monitoring, inter-agent delegation, and memory management—have *no* OTel representation, rendering them as opaque `INTERNAL` spans. This semantic gap makes automated fault detection impossible for failure modes such as reasoning loops, circular delegation, guardrail bypass, and planning failures.

We present AgentTelemetry, an open-source Python toolkit that fills this gap. The tool makes three contributions:

(1) **A grounded span taxonomy** of nine agent-specific span kinds (five novel), derived from systematic analysis of seven production frameworks and validated through ablation.
(2) **A modular analysis toolkit** comprising four analysis modules (anomaly detection, cost aggregation, decision attribution, hallucination tracing) and a real-time circuit breaker.
(3) **Seven framework adapters** using three instrumentation strategies (callback handlers, monkey-patching, manual API), requiring as few as 3 lines of code to instrument an agent.

The toolkit is pip-installable, OTel-native (all spans export via OTLP to any compatible backend), and imposes <0.006% overhead on end-to-end agent execution.

Core Layer
9 Span Kinds · Privacy Filter · Context Propagation · Exporters
Adapter Layer
LangChain · CrewAI · AutoGen · LlamaIndex · Anthropic · OpenAI · Custom
Analysis Layer
AnomalyDetector · CostAggregator · DecisionAttributor · HallucinationTracer
AgentCircuitBreaker (real-time fault interception)

Figure 1: Three-layer architecture with OTel-native export.

2 Tool Architecture

AgentTelemetry comprises three modules (Figure 1): **Core**, **Adapters**, and **Analysis**.

2.1 Core: Span Taxonomy

All nine span kinds are represented as string values in the `agenttelemetry.span.kind` attribute on standard OTel `INTERNAL`

Conference'17, Washington, DC, USA

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

Table 1: Nine agent-specific span kinds. ★ = novel (no OTel GenAI equivalent); ext. = extends OTel; ovlp. = overlaps with OTel.

Span Kind	Key Attributes	
AGENT	agent.name, .framework, .role	ext.
LLM_CALL	llm.model, .tokens, .cost	ovlp.
TOOL_CALL	tool.name, .input, .status	ovlp.
RETRIEVAL	retrieval.source, .doc_count	ovlp.
PLANNING ★	planning.strategy, .step_count	
REASONING ★	reasoning.chain	
GUARD_RAIL ★	guardrail.name, .result	
DELEGATION ★	delegation.src/tgt_agent	
MEMORY ★	memory.operation, .key	

Table 2: Adapter strategies and supported frameworks.

Strategy	Frameworks	LOC
Callback handlers	LangChain, CrewAI, LlamaIndex	903
Monkey-patching	Anthropic, OpenAI, AutoGen	795
Manual API	Custom agents	315

spans, ensuring compatibility with OTLP, Jaeger, and any OTel-compatible backend. Table 1 lists the kinds with their key attributes and novelty status relative to OTel GenAI.

The five novel kinds capture orchestration phases invisible to OTel GenAI: task decomposition (PLANNING), chain-of-thought (REASONING), safety-check outcomes (GUARD_RAIL), inter-agent handoff (DELEGATION), and state management (MEMORY). An ablation study confirms that every kind is necessary: removing any one makes at least one of 14 fault types undetectable.

A three-level privacy filter (NONE, METADATA_ONLY, FULL) controls attribute capture at the span level. The default (METADATA_ONLY) provides full diagnostic signal without exposing prompt or completion content.

2.2 Adapters: Three Instrumentation Strategies

Seven framework adapters implement three strategies (Table 2). **Callback handlers** (LangChain, CrewAI, LlamaIndex; 903 LOC) register event listeners via framework extension APIs. **Monkey-patching** (Anthropic SDK, OpenAI SDK, AutoGen; 795 LOC) wraps key methods at runtime for SDKs without callback systems. **Manual API** (Custom; 315 LOC) provides context managers for explicit instrumentation. All framework imports are lazy (inside `_instrument()`), avoiding import-time dependencies.

2.3 Analysis Modules and Circuit Breaker

Four analysis modules operate on exported span data: **CostAggregator** sums token costs by model, agent, and trace; **DecisionAttributor** links each TOOL_CALL to its parent LLM_CALL and nearby REASONING spans; **AnomalyDetector** detects circular delegation, infinite retries, cost explosion, and context overflow; **HallucinationTracer** flags ungrounded claims via token-overlap heuristics suitable for trace-level screening.

The **AgentCircuitBreaker** is an OTel `SpanProcessor` that intercepts fault patterns in real time. It dispatches on the span kind attribute to check four policies: reasoning loops ($\geq N$ identical tool calls), cost explosion (cumulative cost threshold), context overflow (token growth rate), and delegation cycles (graph cycle detection). Each triggers a configurable callback (log, handler, or exception). This mechanism is *impossible* without agent-specific span kinds: vanilla OTel spans carry no semantic signal for policy dispatch.

3 Usage

Installation.

```
pip install agenttelemetry
```

Basic instrumentation requires three lines of code:

```
from agenttelemetry import AgentTracer
tracer = AgentTracer("my-agent")
with tracer.start_agent_span("researcher"):
    with tracer.start_llm_span("gpt-4o"):
        result = llm.chat(prompt)
    with tracer.start_tool_span("search"):
        data = search_tool(query)
```

Framework adapter (LangChain example):

```
from agenttelemetry.adapters import LangChainAdapter
adapter = LangChainAdapter()
adapter.instrument() # auto-hooks callbacks
agent.run(query)    # spans emitted automatically
```

Real-time fault detection:

```
from agenttelemetry import AgentCircuitBreaker
cb = AgentCircuitBreaker(
    max_reasoning_loops=3,
    max_cost=0.50,
    on_trigger=lambda f: agent.change_strategy()
)
tracer.add_span_processor(cb)
```

Spans export via OTLP to Jaeger, Grafana Tempo, Datadog, or any OTel-compatible backend. A built-in JSON-Lines exporter supports offline analysis.

4 Evaluation

We evaluate AgentTelemetry along four dimensions: fault detection effectiveness, span kind necessity, runtime overhead, and real-world applicability.

4.1 Fault Detection Rate

We define 14 fault types spanning all nine span kinds (e.g., reasoning loop, circular delegation, guardrail bypass, planning failure, cost explosion, hallucination). Each fault is injected at rate 1.0 into mock LLM clients, and we measure binary FDR: 1 if detected, 0 otherwise. Across 2,940 configurations (14 faults × 5 observability conditions × 7 frameworks × 6 models), AgentTelemetry achieves FDR = 1.000 at both privacy levels, versus 0.429 for vanilla OTel and 0.429 for OTel+GenAI (Table 3). The eight undetected faults under vanilla OTel require the five novel span kinds. False positive rate is 0.000 across all 14 detectors on both mock traces (10 runs) and 112 clean SWE-bench traces (3,060 spans).

4.2 Ablation Study

For each of the nine span kinds, we remove all spans of that kind from the exported trace and re-run fault detection. Every span kind is required for at least one fault type: LLM_CALL is the most broadly

Table 3: Fault detection rate by observability condition.

Condition	FDR (14 faults)
No telemetry	0.000
Vanilla OTel	0.429 (6/14)
OTel + GenAI	0.429 (6/14)
AgentTelemetry (metadata)	1.000 (14/14)
AgentTelemetry (full)	1.000 (14/14)

Table 4: Instrumentation overhead (Apple M4 Pro).

Metric	p50	p95	p99
Span latency (µs)	11.7	13.3	28.2
Memory per span (KB)	3.1	3.1	3.2
Throughput (k spans/s)		58–66	

required (4 faults), while each novel kind has exactly one fault type that depends exclusively on it (e.g., `REASONING`→reasoning loop, `PLANNING`→planning failure, `GUARD_RAIL`→guardrail bypass). No kind can be removed without losing detection capability.

4.3 SWE-bench Case Study

On 112 SWE-bench Lite instances (37% of the benchmark, all 12 repositories), we instrument a ReAct coding agent (GPT-4o-mini, 5 tools, 8 iterations max, $2.53 total cost). The agent produces 3,060 spans across all nine kinds.

AgentTelemetry characterizes reasoning loops as the dominant failure mode: 75% of 84 failed instances (95% CI [66%, 82%]) repeatedly called the same tool with similar queries without converging—a pattern *invisible* to vanilla OTel, which cannot distinguish a reasoning step from any other `INTERNAL` span. A telemetry-guided intervention (the circuit breaker injects a strategy-change prompt after ≥3 identical tool calls) improves the patch rate by +11 pp over a no-intervention control (3-seed mean, $p<0.05$), closing the loop from observability through diagnosis to measurable improvement.

4.4 Real LLM Validation

Across 159 runs with 8 models from two providers (OpenAI, Anthropic), all four analysis modules run correctly on production API traces without modification. Two models (GPT-4o-mini, GPT-4.1-nano) triggered infinite-retry alerts in clean runs—genuine model behaviors, not injected faults—demonstrating that AgentTelemetry detects real failure patterns in production.

4.5 Overhead

Table 4 summarizes instrumentation cost measured over 90,000 iterations (10,000 per span kind). Median span creation is 11.7 µs (p99 < 30 µs), adding <0.006% to end-to-end execution given LLM latencies of 500 ms–10 s. Memory is ∼3.1 KB per span. Sustained throughput exceeds 58,000 spans/s, providing >5,000× headroom over typical agent workloads (1–10 spans/s).

Table 5: Comparison with existing tools. ✓ = supported; ○ = partial; – = absent.

Tool	*Multi-fw*	*Span kinds*	*Multi-agent*	*Privacy*	*Fault det.*	*Open-src*	*OTel-native*
LangSmith [6]	–	○	–	–	–	–	–
Langfuse [7]	○	–	–	–	–	✓	✓
OpenLLMetry [8]	✓	–	–	–	–	✓	✓
Datadog LLM [9]	✓	–	–	–	–	–	○
OTel GenAI [10]	✓	○	○	–	–	✓	✓
AGDebugger [5]	–	○	✓	–	○	–	–
MAST [2]	–	–	–	–	–	✓	–
AgentTelemetry	✓	✓	✓	✓	✓	✓	✓

5 Related Work

Table 5 positions AgentTelemetry against representative tools across seven dimensions critical to production agent observability.

Failure taxonomies. MAST [2] identifies 14 multi-agent failure modes from 1,600+ annotated traces; 12/14 (85.7%) are detectable via our span kinds. AgentDebug [3] categorizes failures across four cognitive dimensions. These works characterize *what* goes wrong but provide no runtime detection infrastructure.

Debugging tools. AGDebugger [5] offers interactive visual debugging for multi-agent conversations, validated through a 14-person user study. AgentRx [4] uses LLM-based log analysis for automated diagnosis. These tools operate on unstructured logs; AgentTelemetry provides the standardized, OTel-native trace data such tools could consume.

LLM observability platforms. LangSmith [6] provides deep LangChain integration but no portability to other frameworks. Langfuse [7] and OpenLLMetry [8] provide OTel-compatible tracing but define no agent-specific span kinds—a planning step and a summarization call are indistinguishable. Datadog LLM Observability [9] adds LLM metrics to a commercial stack but cannot detect agent orchestration faults.

OTel GenAI conventions. The OTel GenAI semantic conventions [10] define eight operation types covering LLM calls, retrieval, tools, and agent lifecycle. AgentTelemetry is a strict superset: three kinds overlap, one extends, and five are entirely novel. All spans coexist with GenAI attributes on the same OTLP export pipeline.

6 Conclusion

AgentTelemetry is an open-source toolkit that extends OpenTelemetry with nine agent-specific span kinds, seven framework adapters, four analysis modules, and a real-time circuit breaker. It detects all 14 fault types (FDR = 1.000 vs. 0.429 for vanilla OTel), characterizes real failures on SWE-bench, and imposes negligible overhead (<30 µs per span). The toolkit is available on PyPI (`pip install agenttelemetry`) under Apache-2.0, with documentation and examples at https://github.com/agenttelemetry/agenttelemetry. Future work includes proposing the span taxonomy as an OpenTelemetry Enhancement Proposal and extending the circuit breaker with adaptive sampling for long-running agents.

References

[1] X. Liu, H. Yu, H. Zhang, et al. AgentBench: Evaluating LLMs as Agents. In *Proc. ICLR*, 2024.
[2] MAST: Multi-Agent System Failure Taxonomy. In *Proc. NeurIPS*, 2025.
[3] AgentDebug: Categorizing Failure Modes Across Cognitive Dimensions. *arXiv preprint*, 2025.
[4] AgentRx: Automated Diagnosis of Multi-Agent Systems. Microsoft Research, 2025.
[5] AGDebugger: An Interactive Visual Debugging Interface for Multi-Agent Conversations. In *Proc. CHI*, 2025.
[6] LangChain. LangSmith: LLM Application Observability Platform. https://smith.langchain.com, 2024.
[7] Langfuse: Open-Source LLM Observability. https://langfuse.com, 2024.
[8] OpenLLMetry: Open-Source LLM Telemetry. https://github.com/traceloop/openllmetry, 2024.
[9] Datadog. LLM Observability. https://www.datadoghq.com/product/llm-observability/, 2024.
[10] OpenTelemetry GenAI Semantic Conventions. https://opentelemetry.io/docs/specs/semconv/gen-ai/, 2024.